

# BÀI TẬP VỀ NHÀ — Ôn tập Bài 17 → 21

**Phạm vi ôn tập:** Bài 17 (Angular Material) + Giai đoạn 5 (Signal Forms)

**Nguyên tắc quan trọng:** KHÔNG động vào project Smart Recipe Box. Toàn bộ bài tập làm trên project riêng `bai-tap-tra-sua`.

## CẢNH BÁO — Signal Forms là tính năng THỬ NGHIỆM

API `form()` / `[formField]` được đánh dấu `@experimental 21.0.0` và **có thể đổi hoàn toàn** ở bản Angular sau. Học để biết hướng đi của Angular thì tốt — nhưng **dự án production vẫn dùng Reactive Forms** như Bài 16.

Bộ bài này viết cho **Angular 21.2.x + Material 21.2.x**. Nếu bạn dùng bản khác, hãy tự tra lại API trong `node_modules` trước khi làm.

## YÊU CẦU ĐẶC BIỆT CỦA BỘ NÀY

Đây là bộ bài **cuối cùng** của khóa. Ngoài phần code, sản phẩm cuối phải đạt chuẩn **Material Design 3** với **bố cục kiểu web app của Google** — có Phần C-M3 riêng ở cuối, kèm bảng token màu và mẫu bố cục cụ thể.

 **Material Design 3 là nội dung MỚI, chưa có trong 21 bài đã học.** Nên bộ này có thêm mục  **Bài giảng bổ sung** ngay trước Phần A — đọc nó trước, đừng nhảy thẳng vào câu hỏi.

## Nguyên tắc soạn Phần A

Phần lý thuyết **không hỏi tên directive hay tên module** của Angular Material. Loại kiến thức đó chỉ đúng với một thư viện, đổi sang PrimeNG hay shadcn là bỏ đi.

Thay vào đó, câu hỏi tập trung vào **nguyên tắc mang đi được**: khả năng tiếp cận, design token, kiến trúc form, đánh đổi khi dùng thư viện. Material chỉ đóng vai **ví dụ minh họa**, không phải nội dung cần thuộc.

Tên selector cụ thể nằm ở **sổ tay tra cứu cuối file** — để tra khi làm bài, không phải để học thuộc.

## Chuẩn bị

Nếu bạn ĐÃ làm bộ 13–16

Dùng luôn project cũ. Chỉ cần cài Material (xem bên dưới).

Nếu bạn CHƯA có project

```
cd <thư mục chứa project của bạn>
```

```
ng new bai-tap-tra-sua --style=css --ssr=false --routing=true --skip-tests
```

```
cd bai-tap-tra-sua && ng serve --port 4300
```

## Cài Angular Material

⚠ **Commit Git trước khi chạy.** `ng add` sửa 4 file cùng lúc. Có gì sai thì `git checkout .` là về nguyên trạng.

```
git add -A && git commit -m "truoc khi cai material"
```

```
ng add @angular/material@21
```

Trả lời các câu hỏi:

| Câu hỏi                   | Chọn                  | Vì sao                                         |
|---------------------------|-----------------------|------------------------------------------------|
| Chọn theme màu            | Bất kỳ (đổi sau được) | Ta sẽ tùy chỉnh ở Phần C-M3                    |
| Set up global typography? | <b>No</b>             | Chọn Yes sẽ ghi đè cỡ chữ <code>h1/h2/p</code> |
| Include animations?       | <b>Yes</b>            | Cần cho hiệu ứng gọn sóng                      |

### Sau khi cài, kiểm tra 3 việc:

1. **Khởi động lại `ng serve`** — `angular.json` vừa đổi, mà server chỉ đọc file này lúc khởi động. Không restart thì theme **không bao giờ được nạp**.
2. Mở `src/styles.css` — nếu Material chèn khối `body` đè lên của bạn, đưa khối của bạn xuống dưới cùng.
3. Mở `src/app/app.config.ts` — nếu thấy `provideAnimationsAsync()` thì xóa, Material v19+ không cần nữa.

## Code khởi đầu (trạng thái tương đương "đã xong Bài 16")

Bỏ qua nếu bạn đã làm bộ 13–16.

- ▶  `src/app/models.ts`
- ▶  `src/app/mock-drinks.ts`
- ▶  `src/app/drink-service.ts` — tạo bằng `ng g service drink-service`
- ▶  `src/app/app.routes.ts` + `app.ts` + `app.html`
- ▶  `drink-list` + `drink-detail` + `add-drink`

# BÀI GIẢNG BỔ SUNG — Material Design 3 là gì?

Phần này **chưa có trong 21 bài đã học**. Đọc trước khi làm Phần A và Phần C-M3.

## Design system là gì?

Khi một công ty có hàng chục sản phẩm, họ cần chúng **trông như cùng một nhà**. Google có Gmail, Drive, Calendar, Maps... và bạn nhìn phát là biết ngay của Google.

Thứ đảm bảo điều đó gọi là **design system** — một bộ quy tắc thống nhất về màu sắc, kiểu chữ, khoảng cách, bo góc. **Material Design** là design system của Google, hiện ở phiên bản **3** (còn gọi là M3, ra mắt 2021).

Mọi công ty lớn đều có: Apple có *Human Interface Guidelines*, Microsoft có *Fluent*, IBM có *Carbon*. Nguyên lý giống nhau, chỉ khác chi tiết.

## Ý tưởng cốt lõi: **design token**

Đây là khái niệm quan trọng nhất, và nó **dùng được ở mọi design system**, không riêng Material.

Thay vì viết giá trị cụ thể:

```
color: #93401f;  
border-radius: 16px;  
font-size: 1.75rem;
```

bạn viết **tên vai trò**:

```
color: var(--mat-sys-primary);  
border-radius: var(--mat-sys-corner-large);  
font: var(--mat-sys-title-large);
```

Khác biệt không phải chuyện gõ dài hay ngắn, mà là **ý nghĩa**:

| Cách viết              | Nói lên điều gì                                          |
|------------------------|----------------------------------------------------------|
| #93401f                | "màu nâu cam này"                                        |
| var(--mat-sys-primary) | " <b>màu chính</b> của thương hiệu, bất kể nó là màu gì" |

Ba lợi ích thực tế:

1. **Đổi một chỗ, cả app đổi theo**. Sếp muốn đổi màu thương hiệu → sửa một dòng trong file theme, không phải đi tìm 200 chỗ có mã hex.
2. **Chế độ tối hoạt động miễn phí**. `--mat-sys-surface` tự trả về màu trắng ở theme sáng và màu than ở theme tối. Bạn không viết thêm CSS nào.

3. **Người sau đọc code hiểu được ý đồ.** `--mat-sys-error` nói rõ "đây là màu báo lỗi", còn `#d32f2f` thì phải đoán.

📌 **Đây là kiến thức mang đi được.** Đổi sang Tailwind (`bg-primary`), sang shadcn (`--primary`), sang bất kỳ hệ nào — ý tưởng y hệt, chỉ khác tên biến.

## Điểm mới lớn nhất của M3: phân tầng bằng **màu nền**, không bằng **bóng đổ**

Material 2 (và Bootstrap, và hầu hết web cũ) tạo cảm giác "nổi lên" bằng **bóng đổ**:

```
box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2); /* cách của M2 */
```

Material 3 chuyển sang dùng **độ đậm nhạt của nền**. Thứ nào "cao hơn" thì nền đậm hơn một chút. Có **năm mức**:

```
surface-container-lowest  ← nhạt nhất, "thấp" nhất
surface-container-low
surface-container         ← mức mặc định
surface-container-high
surface-container-highest ← đậm nhất, "cao" nhất
```

**Vì sao đổi?** Ba lý do:

- **Chế độ tối.** Bóng đổ trên nền đen thì gần như vô hình. Nền đậm nhạt thì thấy rõ ở cả hai chế độ.
- **Dịu mắt.** Một trang đầy bóng đổ nhìn rất "ồn ào".
- **Hiệu năng.** Đổ bóng tốn tài nguyên vẽ hơn tô nền.

Áp dụng thực tế:

```
Nền trang           → --mat-sys-surface
Card nội dung       → --mat-sys-surface-container-low
Thanh bên / menu    → --mat-sys-surface-container-high
Ô nhập liệu         → --mat-sys-surface-container-highest
```

## Cặp màu "nền và chữ trên nền"

M3 luôn đi theo cặp: mỗi màu nền có một màu chữ tương ứng, **đã được đảm bảo đủ tương phản**.

| Nền                                      | Chữ trên nền đó                             |
|------------------------------------------|---------------------------------------------|
| <code>--mat-sys-surface</code>           | <code>--mat-sys-on-surface</code>           |
| <code>--mat-sys-primary</code>           | <code>--mat-sys-on-primary</code>           |
| <code>--mat-sys-primary-container</code> | <code>--mat-sys-on-primary-container</code> |
| <code>--mat-sys-error</code>             | <code>--mat-sys-on-error</code>             |

Tiền tố **on**— nghĩa là "màu để đặt **LÊN TRÊN** nền kia". Dùng đúng cặp thì bạn **không bao giờ phải lo chuyện tương phản WCAG** — Google đã tính sẵn.

⚠ Đây là lý do **không được trộn** — đặt `--mat-sys-on-primary` lên nền `--mat-sys-surface` là sai cặp, và rất dễ ra chữ trắng trên nền trắng.

## Lưới 8dp

Mọi khoảng cách trong Material là **bội số của 8**: 8, 16, 24, 32, 40... Ngoại lệ duy nhất là 4px cho khe rất hẹp.

Vì sao? Vì các con số cùng chia hết cho một số thì **mắt thấy có nhịp điệu**. Còn 13px chỗ này, 19px chỗ kia thì trông "lệch" mà người xem không chỉ ra được tại sao.

💡 Nguyên tắc này cũng dùng được ở mọi design system — Tailwind cũng dùng thang 4px (`p-2` = 8px, `p-4` = 16px).



## PHẦN A — Lý thuyết

Các câu hỏi ở đây tập trung vào **nguyên tắc mang đi được** — thứ bạn vẫn dùng khi đổi sang React, Vue, hay một thư viện UI khác. Không có câu nào bắt học thuộc tên directive của Angular Material.

## Về thư viện UI nói chung

- Suốt Bài 1–16 bạn tự viết CSS. Bài 17 bạn chuyển sang Material. Nêu **ba thứ bạn được** và **hai thứ bạn mất** khi dùng thư viện UI.
- `ng add` khác `npm install` ở điểm nào? Vì sao việc "tự cấu hình hệ" lại có ích **và** có rủi ro?
- Vì sao nên import từng module cụ thể (`@angular/material/button`) thay vì cả gói? Điều này liên quan gì tới thời gian tải trang của người dùng?
- Ở Bài 17 bạn từng đặt `<mat-icon>` vào một khe mà thư viện dành cho **chữ**, và icon biến mất mà **không có lỗi nào báo ra**. Rút ra bài học gì về cách làm việc với thư viện UI nói chung? Khi gặp tình huống "trông sai nhưng không báo lỗi", bạn tra ở đâu?

## Về khả năng tiếp cận (*dùng được ở mọi framework*)

- Một nút có chữ hiển thị "**Thêm món mới**" mà lại gắn `aria-label="Add item"`. Người dùng trình đọc màn hình nghe thấy gì? Rút ra quy tắc khi nào **nên** và khi nào **không nên** dùng `aria-label`.
- Vì sao bọc một biểu tượng **chỉ để trang trí** trong thẻ `<button>` là sai? Nêu **hai** hậu quả cụ thể với người dùng.
- Một dòng trong danh sách bấm chuột thì chuyển trang được, nhưng nhấn **Tab** thì con trỏ nhảy qua. Nguyên nhân gốc là gì? (Gợi ý: nghĩ về **loại thẻ HTML** chứ không phải thư viện.)

8. Ô nhập liệu cần được "nối" với nhãn của nó. Nêu **hai cách** làm điều đó, và cho biết vì sao để thư viện lo hộ thì an toàn hơn tự nối tay.

## Về Signal Forms (*kiến trúc, không phải cú pháp*)

9. Reactive Forms **giữ bản sao** dữ liệu bên trong form. Signal Forms thì **bọc quanh** dữ liệu gốc của bạn. Nêu **hai hệ quả thực tế** của khác biệt này khi bạn viết hàm `save()`.
10. `submit()` tự động làm **ba việc** trước khi chạy hành động của bạn. Kể ra, và giải thích vì sao để thư viện lo ba việc đó tốt hơn tự viết tay.
11. Gọi `reset()` mà ô input **vẫn còn chữ**. Vì sao lại thiết kế như vậy? (Gợi ý: liên hệ với câu 9 — ai là chủ sở hữu dữ liệu?)
12. Vì sao chỉ nên hiện thông báo lỗi khi ô vừa sai **vừa đã được người dùng chạm vào**? Mô tả trải nghiệm tệ nếu bỏ điều kiện thứ hai.
13. `<mat-error>` của Material vốn sinh ra cho Reactive Forms, vậy mà chạy được với Signal Forms. Cơ chế nào cho phép hai thứ khác thể hệ nói chuyện được với nhau? Kỹ thuật này có tên gọi chung là gì trong lập trình?
14. Trong `@for`, vì sao `track` bằng **chính đối tượng** lại gây dựng lại DOM, còn `track` bằng một **chuỗi ổn định** thì không? Liên hệ với bài học `track` ở Bài 11.

## Về design token (*dùng được ở mọi design system*)

15. So sánh `color: #93401f` và `color: var(--mat-sys-primary)`. Nêu **ba lợi ích** của cách thứ hai.
16. Material 3 phân tầng giao diện bằng **độ đậm nhạt của nền** thay vì **bóng đổ** như Material 2. Nêu **hai lý do** khiến Google đổi hướng.
17. Tiền tố `on-` trong `--mat-sys-on-surface` nghĩa là gì? Vì sao dùng đúng cặp nền/chữ thì bạn không phải tự kiểm tra độ tương phản WCAG?
18. Vì sao nên dùng `padding: 16px` thay vì `padding: 13px`? Nguyên tắc này có tên gì, và bạn có gặp lại nó ở Tailwind không?

## PHẦN B — Tìm lỗi trong code

Mỗi đoạn có **đúng một lỗi**. Chỉ ra lỗi, giải thích **tại sao sai**, viết lại cho đúng.

### Lỗi số 1

```
import { MatButtonModule } from '@angular/material/button';

@Component({
  selector: 'app-drink-list',
  imports: [],
```

```

    templateUrl: './drink-list.html',
    styleUrls: ['./drink-list.css'],
  })
  export class DrinkList {}

```

```
<button matButton="filled">Lưu</button>
```

## Lỗi số 2

```

<button matButton="filled" aria-label="Example icon button with a plus
icon">
  <mat-icon>add</mat-icon>
  Thêm món mới
</button>

```

## Lỗi số 3

```

<mat-toolbar>
  <button matIconButton>
    <mat-icon>local_cafe</mat-icon>
  </button>
  <span>Quầy Trà Sữa</span>
</mat-toolbar>

```

## Lỗi số 4

```

<mat-nav-list>
  @for (drink of drinks(); track drink.id) {
    <mat-list-item [routerLink]="['/drinks', drink.id]">
      <a matListItemTitle>{{ drink.name }}</a>
    </mat-list-item>
  }
</mat-nav-list>

```

## Lỗi số 5

```

import { form, Field } from '@angular/forms/signals';

@Component({
  selector: 'app-add-drink',
  imports: [Field],
  templateUrl: './add-drink.html',

```

```
    styleUrls: ['./add-drink.css'],  
  })  
  export class AddDrink {}
```

### Lỗi số 6

```
import { form, FormBuilder } from '@angular/forms';
```

### Lỗi số 7

```
<input matInput [field]="drinkForm.name" />
```

### Lỗi số 8

```
<form [formRoot]="drinkForm" (submit)="save($event)">  
  <button type="submit">Lưu</button>  
</form>
```

```
protected async save(event: Event): Promise<void> {  
  event.preventDefault();  
  await submit(this.drinkForm, async () => {  
    this.drinkService.addDrink(/* ... */);  
  });  
}
```

### Lỗi số 9

```
const ok = await submit(this.drinkForm, async () => {  
  this.drinkService.addDrink(/* ... */);  
});  
  
this.drinkForm().reset();  
this.drinkModel.set({ name: '', description: '', giaCoBan: 0 });
```

### Lỗi số 10

```
protected async save(event: Event): Promise<void> {  
  event.preventDefault();
```



```
await submit(this.drinkForm);
}
```

### Lỗi số 11

```
@for (error of drinkForm.name().errors(); track error) {
  <mat-error>{{ error.message }}</mat-error>
}
```

### Lỗi số 12

```
protected readonly drinkForm = form(this.drinkModel, (path) => {
  required(path.name, 'Tên món là bắt buộc.');
```

## PHẦN C — Project: Quầy Trà Sữa

### Nhiệm vụ C17 — ôn Bài 17: Angular Material

Chuyển **toàn bộ** giao diện sang Material, làm lần lượt từng component: **App** → **DrinkList** → **DrinkDetail** → **AddDrink**.

#### Bản đồ chuyển đổi:

| Hiện tại             | Material thay thế                  | Module                              |
|----------------------|------------------------------------|-------------------------------------|
| <h1> trong app.html  | <mat-toolbar>                      | MatToolbarModule                    |
| Ô tìm kiếm + <label> | <mat-form-field> + matInput        | MatFormFieldModule + MatInputModule |
| <ul> danh sách món   | <mat-nav-list> + <a mat-list-item> | MatListModule                       |
| Biểu tượng 🔥         | <mat-icon>                         | MatIconModule                       |
| Link "Thêm món mới"  | <a matButton="filled">             | MatButtonModule                     |
| Khối chi tiết món    | <mat-card>                         | MatCardModule                       |
| Nút − +              | <button matIconButton>             | MatButtonModule                     |
| Danh sách topping    | <mat-list> + <mat-list-item>       | MatListModule                       |

| Hiện tại            | Material thay thế                              | Module                                           |
|---------------------|------------------------------------------------|--------------------------------------------------|
| 3 ô nhập trong form | <code>&lt;mat-form-field&gt; + matInput</code> | <code>MatFormFieldModule + MatInputModule</code> |
| Nút "Lưu món"       | <code>&lt;button matButton="filled"&gt;</code> | <code>MatButtonModule</code>                     |

Ba thứ **BẮT BUỘC** giữ nguyên dù giao diện đổi thế nào:

- Mọi `aria-label` trên nút chỉ có biểu tượng
- `aria-live="polite"` ở ô hiển thị số ly
- `<a>` vẫn là `<a>` cho điều hướng, `<button type="submit">` vẫn là button cho form

**Dọn CSS:** cái gì Material lo rồi thì xóa. Giữ lại CSS bố cục riêng của bạn.

 **Tự kiểm tra:** nhấn **Tab** liên tục ở trang `/drinks` — con trỏ phải **dừng ở từng tên món**. Nếu nó nhảy qua hết, bạn đã mắc đúng Lỗi số 4 ở Phần B.

## Nhiệm vụ C18 — ôn Bài 18: Signal Forms

**Chuẩn bị:** thêm `authorEmail: string;` vào `DrinkModel`, rồi điền giá trị cho cả 3 món trong `mock-drinks.ts`.

**Yêu cầu:** Thay toàn bộ Reactive Forms trong `AddDrink` bằng Signal Forms, và thêm ô thứ tư cho email người tạo.

Component cần một **signal thường** giữ dữ liệu form với bốn trường, và một form được tạo **bọc quanh** signal đó. Mọi dấu vết `FormBuilder`, `ReactiveFormsModule`, `Validators` phải biến mất — kể cả trong mảng `imports`.

Template: thẻ `<form>` nối với form mới, bốn ô input nối với bốn field. Giữ nguyên `mat-form-field` và `mat-label` của Material.

Hàm `save()` **chưa cần kiểm tra hợp lệ** ở bài này. Nhờ đặc tính "form không giữ bản sao", việc lấy dữ liệu ra sẽ đơn giản bất ngờ.

 Ô giá là số. Chú ý kiểu dữ liệu trong signal model — khởi tạo bằng `0` chứ không phải `''`.

## Nhiệm vụ C19 — ôn Bài 19: Gửi và reset

Thẻ `<form>` **thời dùng** `[formRoot]` — tự bắt sự kiện gửi, tự chặn hành vi mặc định, thêm thuộc tính tắt kiểm tra tự động của trình duyệt.

Hàm `save()` thành bất đồng bộ, bọc phần thêm món vào tiện ích gửi form. Sau khi gửi **thành công**, form phải được dọn sạch — nhớ cần **hai bước**. Gửi thất bại thì dữ liệu người dùng còn nguyên.

**Đừng điều hướng đi đâu cả** — ở lại trang để bạn nhìn thấy form được dọn.

Nút Lưu **tự khóa** khi đang gửi.

 **Phép thử:** bấm nút Lưu thật nhanh 5 lần. Chỉ **một** món được thêm.

## Nhiệm vụ C20 — ôn Bài 20: Validation

Bổ sung hàm mô tả luật vào `form()`:

| Ô                        | Luật                                      |
|--------------------------|-------------------------------------------|
| <code>name</code>        | Bắt buộc + tối thiểu 3 ký tự              |
| <code>description</code> | Bắt buộc                                  |
| <code>giaCoBan</code>    | Bắt buộc + tối thiểu 1000 + tối đa 200000 |
| <code>authorEmail</code> | Bắt buộc + đúng định dạng email           |

Mỗi luật có câu thông báo tiếng Việt rõ nghĩa.

**Không cần sửa gì trong `save()` — `submit()` tự chặn khi form không hợp lệ.**

 **Tự kiểm tra:** bấm Lưu khi chưa điền gì → quay lại danh sách, **không có món rác nào.**

## Nhiệm vụ C21 — ôn Bài 21: Hiện thị lỗi

Bốn ô nhập phải tự báo lỗi. Mỗi ô hiển thị **tất cả** thông báo đang áp lên nó — ô email và ô giá có nhiều luật nên có thể mang nhiều lỗi cùng lúc.

Dùng thẻ Material dành riêng cho thông báo lỗi, để thư viện tự quyết định thời điểm hiện.

Thêm cho ô email một dòng **gợi ý** hiển thị lúc bình thường (Material có thẻ riêng, tên rất giống thẻ báo lỗi).

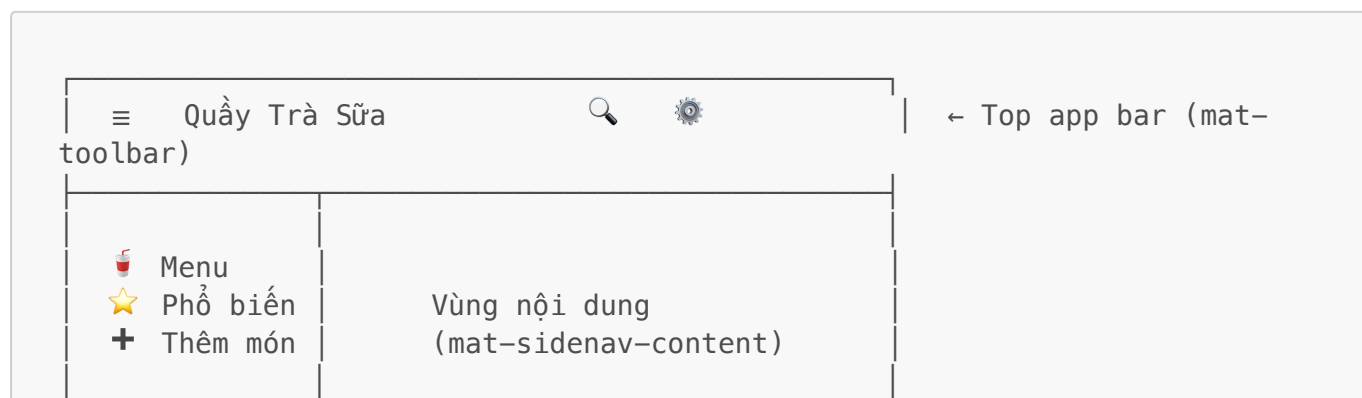
 **Tự kiểm tra:** lúc mới vào trang **không có chữ đỏ nào**, dù cả bốn ô đều trống. Bấm Lưu → cả bốn ô đỏ cùng lúc. Bấm vào một ô rồi bấm ra ngoài → chỉ ô đó đỏ.

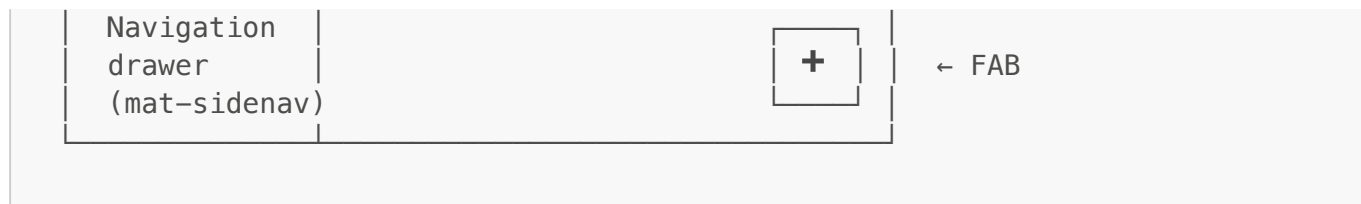
## PHẦN C-M3 — Thiết kế Material Design 3 kiểu Google

Đây là phần **đặc biệt** của bộ bài này. Sau khi xong C17–C21, ứng dụng đã dùng Material. Giờ ta làm cho nó **trông đúng như một sản phẩm của Google**.

### Bố cục chuẩn của web app Google

Mở Gmail, Google Drive, hay Google Keep lên xem — chúng đều theo cùng một khung:





### Yêu cầu bố cục:

1. **Top app bar** — `<mat-toolbar>` cố định trên cùng, chứa nút `≡` mở drawer, tên ứng dụng, và (tùy chọn) nút hành động bên phải.
2. **Navigation drawer** — `<mat-sidenav-container>` bọc toàn bộ, bên trong có `<mat-sidenav>` (menu điều hướng) và `<mat-sidenav-content>` (nội dung). Dùng `<mat-nav-list>` bên trong drawer.
  - Màn hình rộng: drawer **luôn hiện** (`mode="side" + opened`)
  - Màn hình hẹp: drawer **trượt đè lên** (`mode="over"`)
3. **FAB** — `<button matFab>` ở góc dưới bên phải cho hành động chính "Thêm món". Đây là dấu hiệu nhận biết rõ nhất của Material Design.
4. **Danh sách món** dạng **card lưới** thay vì list dọc, mỗi card có ảnh, tên, giá.

Cần thêm hai module: `MatSidenavModule` (`@angular/material/sidenav`) và `MatDividerModule` (`@angular/material/divider`).

## Hệ thống màu: 5 mức bề mặt của M3

Material 3 **không dùng bóng đổ để phân tầng** như M2, mà dùng **độ đậm nhạt của nền**. Tôi đã tra trong bản build — bạn có sẵn đúng 5 mức:

| Token                                            | Dùng cho                         |
|--------------------------------------------------|----------------------------------|
| <code>--mat-sys-surface-container-lowest</code>  | Nhạt nhất — thẻ nổi trên nền tối |
| <code>--mat-sys-surface-container-low</code>     | Card nội dung                    |
| <code>--mat-sys-surface-container</code>         | <b>Nền mặc định của các khối</b> |
| <code>--mat-sys-surface-container-high</code>    | Drawer, menu, khối nổi hơn       |
| <code>--mat-sys-surface-container-highest</code> | Ô nhập liệu, khối nổi nhất       |

Cùng với:

| Nhóm          | Token                                                                                      |
|---------------|--------------------------------------------------------------------------------------------|
| Nền trang     | <code>--mat-sys-surface</code>                                                             |
| Chữ trên nền  | <code>--mat-sys-on-surface</code> (chính), <code>--mat-sys-on-surface-variant</code> (phụ) |
| Màu nhấn      | <code>--mat-sys-primary</code> , <code>--mat-sys-on-primary</code>                         |
| Khối nhấn nhẹ | <code>--mat-sys-primary-container</code> , <code>--mat-sys-on-primary-container</code>     |

| Nhóm | Token                                                     |
|------|-----------------------------------------------------------|
| Viền | <code>--mat-sys-outline, --mat-sys-outline-variant</code> |
| Lỗi  | <code>--mat-sys-error, --mat-sys-error-container</code>   |

**Quy tắc bắt buộc của phần này: KHÔNG được viết một mã màu hex nào.** Toàn bộ màu phải lấy từ token `--mat-sys-*`. Lý do: đổi theme một dòng là cả app đổi theo, và chế độ tối hoạt động miễn phí.

## Bo góc: dùng token, không dùng số

M3 có thang bo góc riêng. Bạn có sẵn:

```
var(--mat-sys-corner-extra-small) /* 4px */
var(--mat-sys-corner-small)      /* 8px */
var(--mat-sys-corner-medium)     /* 12px */
var(--mat-sys-corner-large)      /* 16px – dùng cho card */
var(--mat-sys-corner-extra-large) /* 28px – dùng cho khối lớn */
var(--mat-sys-corner-full)       /* tròn hoàn toàn – dùng cho chip, FAB */
/*
```

## Kiểu chữ: dùng token, không tự đặt cỡ

```
font: var(--mat-sys-display-small); /* tiêu đề trang lớn */
font: var(--mat-sys-headline-medium); /* tiêu đề mục */
font: var(--mat-sys-title-large); /* tên món trong card */
font: var(--mat-sys-body-medium); /* nội dung thường */
font: var(--mat-sys-label-small); /* nhãn nhỏ, chú thích */
```

💡 Đây là thuộc tính `font` viết tắt — nó set một lượt cả `font-family`, `size`, `weight`, `line-height`. Không cần khai từng cái.

## Khoảng cách: lưới 8dp

Material Design đo mọi khoảng cách theo bội số của **8px**: **8px**, **16px**, **24px**, **32px**. Ngoại lệ duy nhất là **4px** cho khoảng cách rất nhỏ.

Đừng dùng **13px**, **19px**, **27px** — nhìn sẽ "lệch" mà khó chỉ ra tại sao.

## Đổi bảng màu chủ đề

Trong `src/material-theme.scss`:

```
color: (primary: mat.$orange-palette, tertiary: mat.$rose-palette);
```

Các palette có sẵn: `$red`, `$green`, `$blue`, `$yellow`, `$cyan`, `$magenta`, `$orange`, `$chartreuse`, `$spring-green`, `$azure`, `$violet`, `$rose` (thêm hậu tố `-palette`).

## Thử thách: chế độ tối

Trong `material-theme.scss`, đổi `color-scheme: light` thành `light dark` và bật chế độ tối trong cài đặt hệ điều hành. **Nếu bạn tuân thủ quy tắc "không hex"**, toàn bộ ứng dụng sẽ chuyển sang tông tối mà bạn **không sửa thêm một dòng CSS** nào.

Nếu có chỗ nào chữ trắng trên nền trắng — chỗ đó chính là nơi bạn đã lỡ viết mã màu cứng. Đây là **cách kiểm tra nhanh nhất** xem bạn đã dùng token đúng chưa.

---

## PHẦN D — Thử thách nâng cao (BẮT BUỘC)

---

- Drawer** đáp ứng theo màn hình. Dùng `BreakpointObserver` từ `@angular/cdk/layout` để tự đổi `mode="side" ↔ mode="over"` theo chiều rộng cửa sổ.
- Chip** lọc theo trạng thái. Dùng `MatChipsModule` thêm hàng chip "Tất cả / Phổ biến" phía trên danh sách.
- Snackbar** xác nhận. Dùng `MatSnackBar` hiện thông báo "Đã thêm món" ở đáy màn hình sau khi lưu thành công.
- Hộp thoại** xác nhận xóa. Dùng `MatDialog` hỏi lại trước khi xóa một món.
- Skeleton** khi tải. Dùng `MatProgressSpinner` hoặc `MatProgressBar` — sẽ rất hữu ích khi bạn học tới `HttpClient`.
- Sắp xếp và lọc kết hợp**. Thêm `MatSelect` chọn tiêu chí sắp xếp (giá tăng/giảm), kết hợp với ô tìm kiếm bằng một `computed` duy nhất.

---

## Bảng tự theo dõi tiến độ

---

| Phần                                                                                | Nội dung                                                     | Đã xong?                 |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------|
|  | Đọc bài giảng bổ sung về Material Design 3                   | <input type="checkbox"/> |
| A                                                                                   | 18 câu lý thuyết                                             | <input type="checkbox"/> |
| B                                                                                   | 12 đoạn code tìm lỗi                                         | <input type="checkbox"/> |
| C17                                                                                 | Chuyển toàn bộ sang Material                                 | <input type="checkbox"/> |
| C18                                                                                 | Signal Forms: <code>form()</code> + <code>[formField]</code> | <input type="checkbox"/> |
| C19                                                                                 | <code>submit()</code> + reset hai bước                       | <input type="checkbox"/> |
| C20                                                                                 | Validators + thông báo tiếng Việt                            | <input type="checkbox"/> |

| Phần  | Nội dung                                                       | Đã xong?                 |
|-------|----------------------------------------------------------------|--------------------------|
| C21   | <code>&lt;mat-error&gt;</code> + <code>&lt;mat-hint&gt;</code> | <input type="checkbox"/> |
| M3-1  | Top app bar + navigation drawer                                | <input type="checkbox"/> |
| M3-2  | FAB cho hành động chính                                        | <input type="checkbox"/> |
| M3-3  | Danh sách dạng card lưới                                       | <input type="checkbox"/> |
| M3-4  | Toàn bộ màu dùng token, <b>không hex</b>                       | <input type="checkbox"/> |
| M3-5  | Bo góc + kiểu chữ dùng token                                   | <input type="checkbox"/> |
| M3-6  | Khoảng cách theo lưới 8dp                                      | <input type="checkbox"/> |
| M3-7  | 🌙 Chế độ tối hoạt động                                         | <input type="checkbox"/> |
| D1–D6 | Thử thách nâng cao                                             | <input type="checkbox"/> |

## Sổ tay tra cứu nhanh (Bài 17 → 21)

### Angular Material — selector đã xác minh từ bản 21.2.x

```

<!-- Nút: 5 kiểu -->
<button matButton>Text</button>
<button matButton="outlined">Outlined</button>
<button matButton="filled">Filled – hành động chính</button>
<button matButton="elevated">Elevated</button>
<button matButton="tonal">Tonal</button>

<!-- Nút chỉ có icon – BẮT BUỘC có aria-label -->
<button matIconButton aria-label="Tăng số ly">
  <mat-icon>add</mat-icon>
</button>

<!-- FAB – hành động chính của cả trang -->
<button matFab aria-label="Thêm món"><mat-icon>add</mat-icon></button>
<button matMiniFab aria-label="Sửa"><mat-icon>edit</mat-icon></button>

<!-- matButton dùng được trên <a> – giữ đúng ngữ nghĩa điều hướng -->
<a matButton="filled" routerLink="/drinks/new">Thêm món</a>

```

```

<!-- Ô nhập – mat-label TỰ nối với input, không cần <label for> -->
<mat-form-field appearance="outline">
  <mat-label>Tên món</mat-label>
  <input matInput type="text" [formField]="drinkForm.name" />
  <mat-icon matSuffix>search</mat-icon>
  <mat-hint>Gợi ý hiện lúc bình thường</mat-hint>
  @for (error of drinkForm.name().errors(); track error.kind) {

```

```

    <mat-error>{{ error.message }}</mat-error>
  }
</mat-form-field>

```

```

<!-- Card -->
<mat-card>
  <img mat-card-image [src]="drink.imgUrl" [alt]="drink.name" />
  <mat-card-header>
    <mat-card-title>{{ drink.name }}</mat-card-title>
    <mat-card-subtitle>{{ drink.description }}</mat-card-subtitle>
  </mat-card-header>
  <mat-card-content>...</mat-card-content>
  <mat-card-actions><button matButton>Chi tiết</button></mat-card-actions>
</mat-card>

```

```

<!-- Danh sách điều hướng - <a> PHẢI là list item để Tab tới được -->
<mat-nav-list>
  @for (drink of drinks(); track drink.id) {
    <a mat-list-item [routerLink]="['/drinks', drink.id]">
      <mat-icon matListItemIcon>local_cafe</mat-icon>
      <span matListItemTitle>{{ drink.name }}</span>
      <span matListItemLine>{{ drink.description }}</span>
      <span matListItemMeta>{{ drink.giaCoBan }}đ</span>
    </a>
  }
</mat-nav-list>

```

| Directive vị trí                                                 | Chỗ đứng                                                  |
|------------------------------------------------------------------|-----------------------------------------------------------|
| <code>matListItemIcon</code> /<br><code>matListItemAvatar</code> | Đầu dòng (trái)                                           |
| <code>matListItemTitle</code>                                    | Dòng chính                                                |
| <code>matListItemLine</code>                                     | Dòng phụ                                                  |
| <code>matListItemMeta</code>                                     | Cuối dòng (phải) — được tạo kiểu như CHỮ, không phải icon |

```

<!-- Bố cục kiểu Google -->
<mat-sidenav-container>
  <mat-sidenav #drawer mode="side" opened>
    <mat-nav-list>
      <a mat-list-item routerLink="/drinks">
        <mat-icon matListItemIcon>local_cafe</mat-icon>
        <span matListItemTitle>Menu</span>
      </a>
    </mat-nav-list>
  </mat-sidenav>
</mat-sidenav-container>

```



```

    </mat-nav-list>
  </mat-sidenav>

  <mat-sidenav-content>
    <mat-toolbar>
      <button matIconButton (click)="drawer.toggle()" aria-label="Mở menu">
        <mat-icon>menu</mat-icon>
      </button>
      <span>Quầy Trà Sữa</span>
    </mat-toolbar>

    <main><router-outlet /></main>
  </mat-sidenav-content>
</mat-sidenav-container>

```

## Đặt icon mặc định một lần cho cả app

```

// app.config.ts
import { MAT_ICON_DEFAULT_OPTIONS } from '@angular/material/icon';

providers: [
  { provide: MAT_ICON_DEFAULT_OPTIONS, useValue: { fontSet: 'material-
symbols-outlined' } },
],

```

Sau đó khởi lập `class="material-symbols-outlined"` ở từng thẻ icon.

## Signal Forms — API đã xác minh từ Angular 21.2.18

```

import {
  form,
  submit,
  FormField,
  FormRoot,
  required,
  email,
  min,
  max,
  minLength,
  maxLength,
  pattern,
} from '@angular/forms/signals'; // ⚠️ '/signals', KHÔNG phải
 '@angular/forms'

@Component({
  imports: [FormField], // ⚠️ FormField là DIRECTIVE. Field là KIỂU –
  không đưa vào đây
})
export class AddDrink {

```

```
// 1. Dữ liệu là signal thường
protected readonly drinkModel = signal({ name: '', giaCoBan: 0,
authorEmail: '' });

// 2. Form bọc QUANH signal – không giữ bản sao
protected readonly drinkForm = form(this.drinkModel, (path) => {
  required(path.name, { message: 'Tên món là bắt buộc.' });
  minLength(path.name, 3, { message: 'Tối thiểu 3 ký tự.' });
  min(path.giaCoBan, 1000, { message: 'Giá tối thiểu 1.000đ.' });
  required(path.authorEmail, { message: 'Email là bắt buộc.' });
  email(path.authorEmail, { message: 'Email không đúng định dạng.' });
});

protected async save(event: Event): Promise<void> {
  event.preventDefault();

  const ok = await submit(this.drinkForm, async () => {
    this.drinkService.addDrink(/* ... đọc thẳng this.drinkModel() ...
*/);
  });

  if (ok) {
    this.drinkForm().reset(); // xóa touched/dirty
    this.drinkModel.set({ name: '', giaCoBan: 0, authorEmail: '' }); //
xóa DỮ LIỆU
  }
}
```

```
<form novalidate (submit)="save($event)">
  <input [formField]="drinkForm.name" />
  <button type="submit" [disabled]="drinkForm().submitting()">Lưu</button>
</form>
```

**Ba tầng dấu ngoặc** — chỗ dễ rối nhất:

```
drinkForm.name; // cái field
drinkForm.name(); // trạng thái của field
drinkForm.name().value(); // giá trị bên trong
```

**Signal trạng thái của mỗi field:**

| Signal                                        | Ý nghĩa               |
|-----------------------------------------------|-----------------------|
| <code>value()</code>                          | Giá trị hiện tại      |
| <code>valid()</code> / <code>invalid()</code> | Hợp lệ / không hợp lệ |

| Signal                 | Ý nghĩa                                                          |
|------------------------|------------------------------------------------------------------|
| <code>touched()</code> | Đã chạm vào rồi bỏ đi                                            |
| <code>dirty()</code>   | Đã bị sửa                                                        |
| <code>errors()</code>  | Mảng lỗi — mỗi lỗi có <code>kind</code> và <code>message?</code> |
| <code>pending()</code> | Đang chờ kiểm tra bất đồng bộ                                    |

Cấp toàn form: `drinkForm().invalid()`, `drinkForm().submitting()`.

## `submit()` tự động làm 3 việc

1. **Chặn gửi trùng** — đang gửi thì lần gọi sau trả `false` ngay, không chạy hành động
2. **`markAllAsTouched()`** — mọi lỗi hiện ra cùng lúc
3. **Chặn khi không hợp lệ** — form sai thì hành động **không bao giờ chạy**

Nên bạn **không cần** viết `if (invalid) return;`.

⚠ `submit()` **bắt buộc có hành động**, thiếu sẽ ném lỗi `NG1915`. Đây là lý do `[formRoot]` (vốn gọi `submit()` không kèm hành động) không dùng chung được với cách tự gọi `submit()`.

## Material Design 3 — token đã xác minh (165 token trong bản build)

```
.card {
  background-color: var(--mat-sys-surface-container-low);
  color: var(--mat-sys-on-surface);
  border-radius: var(--mat-sys-corner-large);
  padding: 16px; /* lưới 8dp */
}

.card-title {
  font: var(--mat-sys-title-large);
  color: var(--mat-sys-on-surface);
}

.card-desc {
  font: var(--mat-sys-body-medium);
  color: var(--mat-sys-on-surface-variant);
}
```

**Năm mức bề mặt** (M3 phân tầng bằng độ đậm nhạt, không bằng bóng đổ):

```
surface-container-lowest ← nhạt nhất
surface-container-low
surface-container        ← mặc định
surface-container-high
surface-container-highest ← đậm nhất
```

**Quy tắc vàng: không viết một mã hex nào.** Kiểm tra bằng cách bật chế độ tối — chỗ nào chữ trắng trên nền trắng là chỗ đó bạn đã viết màu cứng.

## Những cái bẫy hay dính

| Bẫy                                                    | Sai                                                                 | Đúng                                                                                        |
|--------------------------------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Quên module vào <code>imports</code>                   | <code>imports: []</code>                                            | <code>imports: [MatButtonModule]</code>                                                     |
| <code>aria-label</code> che chữ hiển thị               | <code>&lt;button aria-label="Add"&gt;Thêm món&lt;/button&gt;</code> | bỏ <code>aria-label</code>                                                                  |
| Nút giả cho icon trang trí                             | <code>&lt;button matIconButton&gt;<br/>&lt;mat-icon&gt;...</code>   | <code>&lt;mat-icon aria-hidden="true"&gt;</code>                                            |
| List item không tab tối được                           | <code>&lt;mat-list-item [routerLink]&gt;</code>                     | <code>&lt;a mat-list-item [routerLink]&gt;</code>                                           |
| Icon vào khe <code>meta</code> bị mất cỡ               | <code>&lt;mat-icon matListItemMeta&gt;</code>                       | thêm CSS <code>width/height/font-size: 24px</code> , hoặc dùng <code>matListItemIcon</code> |
| Sai đường dẫn import                                   | <code>from '@angular/forms'</code>                                  | <code>from '@angular/forms/signals'</code>                                                  |
| Nhầm kiểu với directive                                | <code>imports: [Field]</code>                                       | <code>imports: [FormField]</code>                                                           |
| Cú pháp cũ                                             | <code>[field]="form.name"</code>                                    | <code>[formField]="form.name"</code>                                                        |
| <code>[formRoot]</code> + tự gọi <code>submit()</code> | cả hai cùng lúc                                                     | bỏ <code>[formRoot]</code> , thêm <code>novalidate</code>                                   |
| <code>submit()</code> không có hành động               | <code>submit(form)</code>                                           | <code>submit(form, async () =&gt; {...})</code>                                             |
| Tường <code>reset()</code> xóa dữ liệu                 | chỉ <code>reset()</code>                                            | <code>reset()</code> và <code>model.set({...})</code>                                       |
| Dọn form ngoài <code>if (ok)</code>                    | luôn dọn                                                            | chỉ dọn khi thành công                                                                      |
| <code>track</code> bằng đối tượng                      | <code>track error</code>                                            | <code>track error.kind</code>                                                               |
| Viết mã màu cứng                                       | <code>color: #93401f</code>                                         | <code>color: var(--mat-sys-primary)</code>                                                  |
| Khoảng cách lề                                         | <code>padding: 13px</code>                                          | <code>padding: 16px</code> (bội số 8)                                                       |